

LAPORAN PRAKTIKUM

PEMROGRAMAN BERBASIS OBJEK

MEMBUAT FUNGSI CRUD (Create, Read, Update, Delete) USER DENGAN DATABASE MYSQL



OLEH:

DEVINA AMANDA PUTRI

(2411533009)

DOSEN PENGAMPU:

NURFIAH, S.ST, M.KOM

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2025

A. PENDAHULUAN

Pemrograman Berorientasi Objek (PBO) merupakan ilmu pemrograman yang berfokus pada konsep objek, di mana setiap objek memiliki atribut dan metode. Ilmu ini membantu developer membangun aplikasi yang lebih terstruktur, modular, dan mudah dikelola. Dalam praktikum kali ini, konsep PBO diterapkan dalam pembuatan aplikasi laundry yang mengimplementasikan operasi dasar CRUD.

CRUD (Create, Read, Update, Delete) merupakan empat operasi dasar dalam manajemen data, operasi ini adalah dasar dalam sebuah aplikasi yang mana operasi ini bisa untuk membuat, membaca, mengubah dan menghapus suatu data pada database aplikasi. Untuk mendukung proses tersebut, digunakan beberapa komponen penting. Xampp berperan sebagai lingkungan pengembangan lokal yang terdiri dari Apache, MySQL, PHP, dan Perl. Dalam konteks PBO berbasis Java, koneksi ke database MySQL dilakukan menggunakan MySQL Connector/J, sebuah driver yang memungkinkan aplikasi berinteraksi langsung dengan database. Selain itu, digunakan pula pendekatan DAO (Data Access Object) untuk memisahkan logika bisnis dengan logika akses data sehingga kode lebih modular, reusable, dan mudah dikembangkan. Konsep Interface dalam Java juga dimanfaatkan untuk mendefinisikan kontrak method yang wajib diimplementasikan oleh class terkait, sehingga struktur program lebih konsisten dan terorganisir.

Dengan menggabungkan konsep PBO dan implementasi CRUD, dapat dipahami cara pengolahan data dalam aplikasi dan juga mempraktikkan bagaimana prinsip modularitas, enkapsulasi, dan reusabilitas diterapkan dalam pembangunan software.

B. TUJUAN

Tujuan praktikum ini yaitu mahasiswa mampu membuat fungsi CRUD data user menggunakan database MySQL, Adapun poin-poin praktikum yaitu :

- Mahasiswa mampu membuat tabel user pada database MySQL
- Mahasiswa mampu membuat koneksi Java dengan database MySQL
- Mahasiswa mampu membuat tampilan GUI CRUD user
- Mahasiswa mampu membuat dan mengimplementasikan interface
- Mahasiswa mampu membuat fungsi DAO (Data Access Object) dan mengimplementasikan nya.
- Mahasiswa mampu membuat fungsi CRUD dengan menggunakan konsep Pemrograman Berorientasi Objek

C. LANGKAH KERJA

1. Menggunakan Fungsi CRUD pada GUI

- a. Membuat method reset pada class UserFrame yang digunakan untuk menghapus inputan save, update dan delete ketika suatu proses berhasil dilakukan.

```
62 */
63 public void reset() {
64     txtName.setText("");
65     txtUsername.setText("");
66     txtPassword.setText("");
67 }
```

- b. Membuat instance pada UserFrame yang berfungsi sebagai persiapan data dan objek agar form dapat berinteraksi dengan database, baik untuk mengambil, menampilkan, maupun memanipulasi data user.

```
33
34 //user_repo
35 UserRepo usr = new UserRepo();
36 List<User> ls;
37 public String id;
38
```

2. Create User

- a. Buka tab desain pada class UserFrame, pilih button save dan klik kanan untuk menambahkan action performed agar data baru dapat tersimpan ke database.

```
104
105 //btn save
106 JButton btnSave = new JButton("Save");
107 btnSave.addActionListener(new ActionListener() {
108     public void actionPerformed(ActionEvent e) {
109         User user = new User();
110         user.setName(txtName.getText());
111         user.setUsername(txtUsername.getText());
112         user.setPassword(txtPassword.getText());
113         usr.save(user);
114
115         JOptionPane.showMessageDialog(null, "Data berhasil disimpan!");
116     }
117 });
118 btnSave.setForeground(new Color(0, 0, 0));
119 btnSave.setBounds(119, 200, 89, 29);
120 contentPane.add(btnSave);
121
```

3. Read User

- a. Membuat method loadTable() yang dipakai untuk memuat ulang data dari database dan menampilkannya pada tabel.

```
70
71 public void loadTable() {
72     ls = usr.show();
73     TableUser tu = new TableUser(ls);
74     tableUsers.setModel(tu);
75     tableUsers.getTableHeader().setVisible(true);
76 }
77
```

- b. Memanggil method loadTable untuk memastikan saat aplikasi pertama kali dijalankan, tabel user langsung menampilkan data dari database

tanpa harus menekan tombol apapun terlebih dahulu.

```
52         }
53         UserFrame frame = new UserFrame();
54         frame.setVisible(true);
55         frame.loadTable();
56     }
57 }
58 }
```

4. Update User

- a. Pada tab design, klik kanan pada.JTable untuk menambahkan event handler yang berfungsi untuk mengambil id user dan menyimpan, lalu mengambil data nama, username, dan password lalu menampilkan ke form inputan.

```
166
167 tableUsers = new JTable();
168 tableUsers.addMouseListener(new MouseAdapter() {
169     @Override
170     public void mouseClicked(MouseEvent e) {
171         id = tableUsers.getValueAt(tableUsers.getSelectedRow(),0).toString();
172         txtName.setText(tableUsers.getValueAt(tableUsers.getSelectedRow(),1).toString());
173         txtUsername.setText(tableUsers.getValueAt(tableUsers.getSelectedRow(),2).toString());
174         txtPassword.setText(tableUsers.getValueAt(tableUsers.getSelectedRow(),3).toString());
175     }
176 }
177 );
178 tableUsers.setBounds(44, 250, 484, 249);
179 contentPane.add(tableUsers);
180 }
181 }
```

Setelah di run dan memilih salah satu table, akan muncul seperti berikut:

frame

Name jeouy

Username jejejoy13

Password itsjejoy

Save Update Delete Cancel

1	jejoy	jejeoy13	itsjejoybruh
2	jeouy	jejejoy13	itsjejoy

- b. Masih pada tab design, klik kanan button update untuk mengubah data user yang sudah ada di database sebelumnya.

```
121 JButton btnUpdate = new JButton("Update");
122 btnUpdate.addActionListener(new ActionListener() {
123     public void actionPerformed(ActionEvent e) {
124         User user = new User();
125         user.setName(txtName.getText());
126         user.setUsername(txtUsername.getText());
127         user.setId(id);
128         usr.update(user);
129         reset();
130         loadTable();
131     }
132 });
133 btnUpdate.setBounds(220, 200, 97, 29);
134 contentPane.add(btnUpdate);
```

5. Delete User

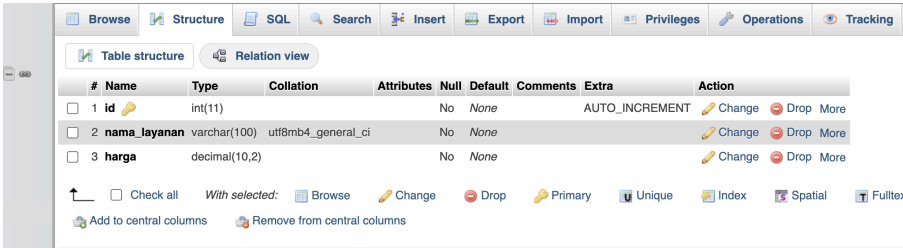
Pada tab design klik kanan pada button delete untuk menambahkan action performed yang bertujuan untuk menghapus data user dari database sesuai ID.

```
136 JButton btnDelete = new JButton("Delete");
137 btnDelete.addActionListener(new ActionListener() {
138     public void actionPerformed(ActionEvent e) {
139         if(id != null) {
140             usr.delete(id);
141             reset();
142         } else {
143             JOptionPane.showMessageDialog(null, "Silahkan pilih data yang akan di hapus");
144         }
145     }
146 });
147 btnDelete.setBounds(329, 200, 97, 29);
148 contentPane.add(btnDelete);
149
```

6. TUGAS (Membuat fungsi CRUD untuk layanan dan Pelanggan)

a. Fungsi CRUD Layanan

1. Membuat tabel database pada xampp



The screenshot shows the XAMPP MySQL database interface. The 'Table structure' tab is selected, displaying the structure of a table named 'layanan'. The table has three columns: 'id' (int(11), primary key, auto-increment), 'nama_layanan' (varchar(100)), and 'harga' (decimal(10,2)). The interface includes various icons for actions like Change, Drop, and More, as well as checkboxes for 'Check all' and 'With selected'.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
2	nama_layanan	varchar(100)	utf8mb4_general_ci		No	None			Change Drop More
3	harga	decimal(10,2)			No	None			Change Drop More

2. Membuat model class layanan (service) dengan attribute id, jenis, harga dan status. Lalu membuatkan setter getter.

```
1 package model;
2
3 public class Service {
4     String id;
5     String jenis;
6     double harga;
7     String status;
8
9     // Getter & Setter
10    public String getId() {
11        return id;
12    }
13    public void setId(String id) {
14        this.id = id;
15    }
16
17    public String getJenis() {
18        return jenis;
19    }
20    public void setJenis(String jenis) {
21        this.jenis = jenis;
22    }
23
24    public double getHarga() {
25        return harga;
26    }
27    public void setHarga(double harga) {
28        this.harga = harga;
29    }
30
31    public String getStatus() {
32        return status;
33    }
34    public void setStatus(String status) {
35        this.status = status;
36    }
37 }
38
```

3. Membuat Interface LayananDAO yang mendefinisikan aturan CRUD pada tabel layanan.

```
1 package DAO;
2
3 import java.util.List;
4 import model.Service;
5
6 public interface LayananDAO {
7     void save(Service layanan);
8     List<Service> show();
9     void update(Service layanan);
10    void delete(String id);
11 }
12
```

4. Membuat LayananRepo untuk mengimplementasikan interface yang sudah dibuat

```

1 package DAO;
2
3 import java.sql.*;
4 import java.util.ArrayList;
5 import java.util.List;
6 import java.util.logging.Level;
7 import java.util.logging.Logger;
8
9 import config.DataBase;
10 import model.Service;
11
12 public class LayananRepo implements LayananDAO {
13     private Connection connection;
14
15     final String insert = "INSERT INTO layanan (nama_layanan, harga) VALUES (?, ?)";
16     final String select = "SELECT * FROM layanan";
17     final String update = "UPDATE layanan SET nama_layanan=?, harga=? WHERE id=?";
18     final String delete = "DELETE FROM layanan WHERE id=?";
19
20     public LayananRepo() {
21         connection = DataBase.koneksi();
22     }
23
24     @Override
25     public void save(Service layanan) {
26         try (PreparedStatement st = connection.prepareStatement(insert)) {
27             st.setString(1, layanan.getJenis());
28             st.setDouble(2, layanan.getHarga());
29             st.executeUpdate();
30         } catch (SQLException e) {
31             Logger.getLogger(LayananRepo.class.getName()).log(Level.SEVERE, null, e);
32         }
33     }
34
35     @Override
36     public List<Service> show() {
37         List<Service> ls = new ArrayList<>();
38         try (Statement st = connection.createStatement(); ResultSet rs = st.executeQuery(select)) {
39             while (rs.next()) {
40                 Service layanan = new Service();
41                 layanan.setId(rs.getString("id"));
42                 layanan.setJenis(rs.getString("nama_layanan"));
43                 layanan.setHarga(rs.getDouble("harga"));
44                 ls.add(layanan);
45             }
46         } catch (SQLException e) {
47             Logger.getLogger(LayananRepo.class.getName()).log(Level.SEVERE, null, e);
48         }
49         return ls;
50     }
51

```

```

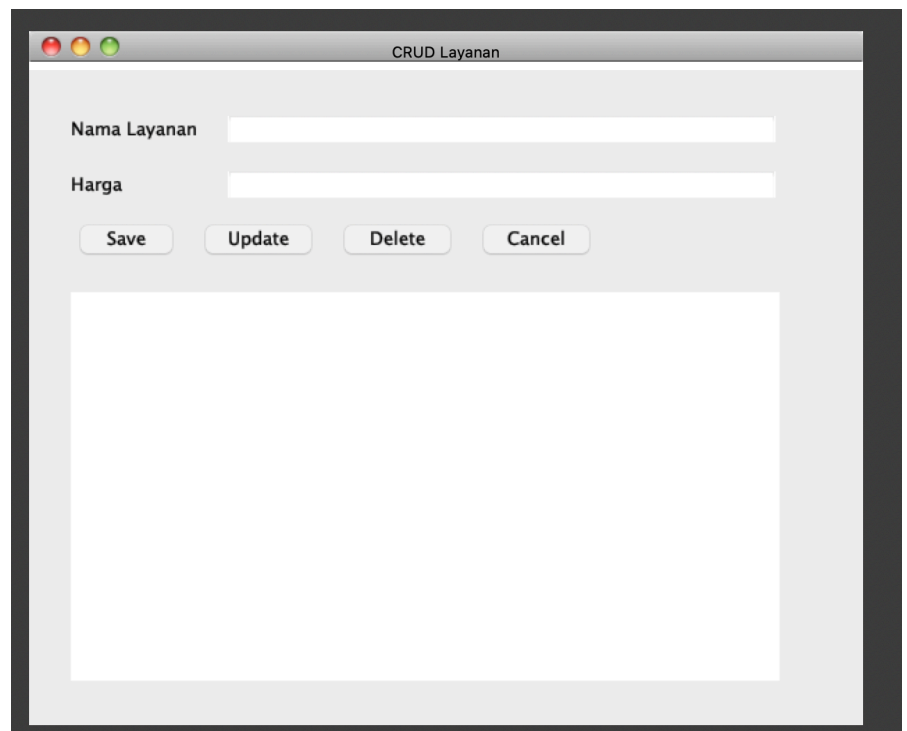
51
52     @Override
53     public void update(Service layanan) {
54         try (PreparedStatement st = connection.prepareStatement(update)) {
55             st.setString(1, layanan.getJenis());
56             st.setDouble(2, layanan.getHarga());
57             st.setString(3, layanan.getId());
58             st.executeUpdate();
59         } catch (SQLException e) {
60             Logger.getLogger(LayananRepo.class.getName()).log(Level.SEVERE, null, e);
61         }
62     }
63
64     @Override
65     public void delete(String id) {
66         try (PreparedStatement st = connection.prepareStatement(delete)) {
67             st.setString(1, id);
68             st.executeUpdate();
69         } catch (SQLException e) {
70             Logger.getLogger(LayananRepo.class.getName()).log(Level.SEVERE, null, e);
71         }
72     }
73 }
74

```

5. Membuat tabel layanan.

```
1 package table;
2
3 import java.util.List;
4 import javax.swing.table.AbstractTableModel;
5 import model.Service;
6
7 public class TableLayanan extends AbstractTableModel {
8     List<Service> ls;
9     private String[] columnNames = {"ID", "Nama Layanan", "Harga"};
10
11     public TableLayanan(List<Service> ls) {
12         this.ls = ls;
13     }
14
15     @Override
16     public int getRowCount() {
17         return ls.size();
18     }
19
20     @Override
21     public int getColumnCount() {
22         return columnNames.length;
23     }
24
25     @Override
26     public String getColumnName(int column) {
27         return columnNames[column];
28     }
29
30     @Override
31     public Object getValueAt(int rowIndex, int columnIndex) {
32         Service layanan = ls.get(rowIndex);
33         switch (columnIndex) {
34             case 0: return layanan.getId();
35             case 1: return layanan.getJenis();
36             case 2: return layanan.getHarga();
37             default: return null;
38         }
39     }
40 }
41
```

6. Membuat tampilan ui untuk crud layanan dengan komponen JTextArea, JButton dan JTable.



D. KESIMPULAN

Dari praktikum ini dapat disimpulkan bahwa Pemrograman Berbasis Objek (PBO) memberikan pemahaman mendasar mengenai pengelolaan data dalam sebuah

aplikasi. Konsep DAO membantu memisahkan logika akses data dari logika bisnis, sementara interface memberi kerangka dasar agar implementasi lebih konsisten. Integrasi dengan MySql Connector memungkinkan aplikasi Java berinteraksi dengan database secara efektif, mulai dari penyimpanan hingga penghapusan data.